

Coding Interview Prep — Complete Detailed Study Guide (English)

For every problem: how to recognize the pattern, an approach hint (not the full solution), and the time/space complexity target. Try each problem yourself first, then compare.

Chapter 1: Arrays

Core pattern: prefix sum, sorting, hashing, in-place manipulation

1. Contains Duplicate

Recognize: “does a duplicate exist” -> Hashing/Set **Hint:** Loop once, inserting each element into a HashSet. If already present, duplicate found. **Target:** Time $O(n)$, Space $O(n)$

2. Valid Anagram

Recognize: matching character frequencies between two strings **Hint:** Use a HashMap/count array — increment for the first string, decrement for the second. Everything at 0 means anagram. **Target:** Time $O(n)$, Space $O(1)$

3. Two Sum

Recognize: two numbers summing to a target **Hint:** HashMap of (value -> index). For each element check if `target - current` was already seen. **Target:** Time $O(n)$, Space $O(n)$

4. Group Anagrams

Recognize: grouping strings that are anagrams of each other **Hint:** Use sorted string (or 26-length count tuple) as HashMap key. **Target:** Time $O(n \cdot k \log k)$, Space $O(n \cdot k)$

5. Top K Frequent Elements

Recognize: “top K” / “most frequent” -> heap or bucket sort **Hint:** Count frequencies with a HashMap, then bucket sort (index = frequency) for $O(n)$. **Target:** Time $O(n)$, Space $O(n)$

6. Product of Array Except Self

Recognize: product except self, no division **Hint:** Two passes — prefix product from left, then multiply in suffix product from right. **Target:** Time $O(n)$, Space $O(1)$ extra

7. Valid Sudoku

Recognize: grid duplicate validation in rows/columns/boxes **Hint:** Three HashSet arrays (rows[9], cols[9], boxes[9]); box index = $(row/3) \cdot 3 + col/3$. **Target:** Time $O(1)$, Space $O(1)$

8. Encode and Decode Strings

Recognize: variable-length string list <-> single string **Hint:** Prefix each string with its length + delimiter, e.g. 5#hello. **Target:** Time $O(n)$, Space $O(n)$

9. Longest Consecutive Sequence

Recognize: longest consecutive run in $O(n)$, no sorting **Hint:** HashSet of all elements; only start counting from a sequence “start” (where `num-1` is absent). **Target:** Time $O(n)$, Space $O(n)$

10. Best Time to Buy and Sell Stock

Recognize: buy once, sell once, max profit **Hint:** Track running minimum price; compare current price minus min against best profit. **Target:** Time $O(n)$, Space $O(1)$

11. Maximum Subarray (Kadane's Algorithm)

Recognize: max sum of contiguous subarray **Hint:** Running sum; reset to 0 if it goes negative; track global max. **Target:** Time $O(n)$, Space $O(1)$

12. Maximum Product Subarray

Recognize: max product of contiguous subarray, negatives flip signs **Hint:** Track currentMax and currentMin at each index; swap them on negative numbers. **Target:** Time $O(n)$, Space $O(1)$

13. Find Minimum in Rotated Sorted Array

Recognize: rotated sorted array -> Binary Search variant **Hint:** Compare `nums[mid]` to `nums[right]` to decide which half holds the minimum. **Target:** Time $O(\log n)$, Space $O(1)$

14. Search in Rotated Sorted Array

Recognize: find target in rotated sorted array **Hint:** At each step identify which half is sorted, then check if target lies in that range. **Target:** Time $O(\log n)$, Space $O(1)$

15. 3Sum

Recognize: three numbers summing to 0 -> sort + two pointers **Hint:** Sort, fix index *i*, two-pointer the rest; skip duplicates. **Target:** Time $O(n^2)$, Space $O(1)$ extra

16. Container With Most Water

Recognize: two pointers from both ends, height-based area **Hint:** Move the pointer at the smaller height inward (it's the bottleneck). **Target:** Time $O(n)$, Space $O(1)$

17. Rotate Array

Recognize: in-place rotation by *k* **Hint:** Reverse whole array, reverse first *k*, reverse the rest. **Target:** Time $O(n)$, Space $O(1)$

18. Merge Intervals

Recognize: merge overlapping intervals **Hint:** Sort by start; merge with last result interval if overlapping. **Target:** Time $O(n \log n)$, Space $O(n)$

19. Insert Interval

Recognize: insert into sorted non-overlapping intervals **Hint:** Three phases — add-before, merge-overlap, add-after. **Target:** Time $O(n)$, Space $O(n)$

20. Non-overlapping Intervals

Recognize: min removals for non-overlap -> Greedy **Hint:** Sort by end time; drop the interval with larger end time on overlap. **Target:** Time $O(n \log n)$, Space $O(1)$

21. Meeting Rooms II

Recognize: max concurrent meetings **Hint:** Sort start/end separately; two-pointer sweep reusing rooms when a meeting ends before another starts. **Target:** Time $O(n \log n)$, Space $O(n)$

22. Set Matrix Zeroes

Recognize: zero out row+column in-place **Hint:** Use first row/column as markers; save their own zero-state in two booleans first. **Target:** Time $O(m \cdot n)$, Space $O(1)$

23. Spiral Matrix

Recognize: boundary tracking **Hint:** Four boundaries (top/bottom/left/right), traverse and shrink each in turn. **Target:** Time $O(m \cdot n)$, Space $O(1)$ extra

24. Rotate Image

Recognize: rotate $n \times n$ matrix 90° in-place **Hint:** Transpose, then reverse each row. **Target:** Time $O(n^2)$, Space $O(1)$

25. Word Search

Recognize: path matching in grid $\rightarrow$ DFS + backtracking **Hint:** DFS from each cell, mark visited temporarily (e.g. #), restore on backtrack. **Target:** Time $O(m \cdot n \cdot 4^L)$, Space $O(L)$

Self-check: Which problems did hashing turn from $O(n^2)$ to $O(n)$? Where did two pointers need sorting first, and where not? What two Binary Search variants did you use on rotated arrays?

Chapter 2: Two Pointers

Core pattern: scanning from both ends of a sorted array/string, fast-slow pointer

1. Valid Palindrome

Recognize: two pointers converge from both ends **Hint:** Skip non-alphanumeric characters, compare lowercase letters as pointers move inward. **Target:** Time $O(n)$, Space $O(1)$

2. Two Sum II - Input Array Is Sorted

Recognize: sorted array two-sum **Hint:** Left/right pointers; move left if sum too small, right if too large. **Target:** Time $O(n)$, Space $O(1)$

3. 3Sum

Recognize: three numbers summing to 0 **Hint:** Sort, fix one index, two-pointer the rest. **Target:** Time $O(n^2)$, Space $O(1)$

4. 3Sum Closest

Recognize: like 3Sum but track closest sum instead of exact match **Hint:** Sort + two pointers, update the closest-difference answer at every comparison. **Target:** Time $O(n^2)$, Space $O(1)$

5. 4Sum

Recognize: four numbers summing to target **Hint:** Sort, fix two outer indices, two-pointer the remaining two; skip duplicates at every level. **Target:** Time $O(n^3)$, Space $O(1)$

6. Container With Most Water

Recognize: two pointers, height bottleneck **Hint:** Move the smaller-height side inward. **Target:** Time $O(n)$, Space $O(1)$

7. Trapping Rain Water

Recognize: water trapped depends on max height on both sides **Hint:** Two pointers with leftMax/rightMax; move the side with the smaller max, add $\text{max} - \text{height}[\text{side}]$ to the total. **Target:** Time $O(n)$, Space $O(1)$

8. Remove Duplicates from Sorted Array

Recognize: in-place dedup on sorted array **Hint:** Slow pointer marks last unique position; fast pointer scans and writes when a new value is found. **Target:** Time $O(n)$, Space $O(1)$

9. Move Zeroes

Recognize: move zeros to end, keep relative order **Hint:** Slow pointer tracks next non-zero write position; fast pointer scans and swaps. **Target:** Time $O(n)$, Space $O(1)$

10. Sort Colors (Dutch National Flag)

Recognize: sort array of 3 distinct values in-place **Hint:** Three pointers (low, mid, high); swap 0s to front, 2s to back, leave 1s in the middle. **Target:** Time $O(n)$, Space $O(1)$

11. Reverse String

Recognize: in-place reversal **Hint:** Left/right pointers swap and move toward center. **Target:** Time $O(n)$, Space $O(1)$

12. Squares of a Sorted Array

Recognize: sorted output from array with negatives **Hint:** Two pointers from both ends comparing absolute values, fill result array from the back. **Target:** Time $O(n)$, Space $O(n)$

13. Boats to Save People

Recognize: pair extremes under a weight limit -> Greedy + two pointers **Hint:** Sort weights; pair heaviest with lightest if they fit together, else send heaviest alone. **Target:** Time $O(n \log n)$, Space $O(1)$

14. Linked List Cycle (fast-slow pointer)

Recognize: detect a cycle **Hint:** Fast pointer moves 2 steps, slow moves 1; if they meet, a cycle exists (Floyd's algorithm). **Target:** Time $O(n)$, Space $O(1)$

15. Middle of the Linked List

Recognize: find middle node in one pass **Hint:** Fast/slow pointer; when fast reaches the end, slow is at the middle. **Target:** Time $O(n)$, Space $O(1)$

Self-check: Which problems needed sorting before two pointers would work? Where did you use 3 pointers instead of 2, and why?

Chapter 3: Stack

Core pattern: LIFO for nested structures or “matching” problems

1. Valid Parentheses

Recognize: matching brackets **Hint:** Push open brackets; on a closing bracket, pop and check it matches. **Target:** Time $O(n)$, Space $O(n)$

2. Min Stack

Recognize: $O(1)$ min retrieval alongside stack operations **Hint:** Auxiliary stack tracking the running minimum, or store (value, currentMin) pairs. **Target:** Time $O(1)$ per op, Space $O(n)$

3. Evaluate Reverse Polish Notation

Recognize: postfix expression evaluation **Hint:** Push operands; on an operator, pop two, compute, push result back. **Target:** Time $O(n)$, Space $O(n)$

4. Generate Parentheses

Recognize: valid combinations under open/close constraints **Hint:** Recursion tree (backtracking): add '(' if open count < n, add ')' if close count < open count. **Target:** Time $O(4^n / \sqrt{n})$, Space $O(n)$

5. Daily Temperatures

Recognize: “next greater element” distance **Hint:** Monotonic decreasing stack of indices; pop and compute distance when a warmer day appears. **Target:** Time $O(n)$, Space $O(n)$

6. Car Fleet

Recognize: grouping by arrival-time comparison -> monotonic stack **Hint:** Sort by position descending, compute time-to-target for each; a new fleet forms only if its time exceeds the fleet ahead. **Target:** Time $O(n \log n)$, Space $O(n)$

7. Largest Rectangle in Histogram

Recognize: area bounded by minimum height -> monotonic stack **Hint:** Monotonic increasing stack of indices; when a shorter bar appears, pop and compute area using popped height and current width. **Target:** Time $O(n)$, Space $O(n)$

8. Next Greater Element I & II

Recognize: next greater value to the right (possibly circular) **Hint:** Monotonic decreasing stack; for the circular version (II), traverse the array twice. **Target:** Time $O(n)$, Space $O(n)$

9. Simplify Path

Recognize: Unix path normalization **Hint:** Split by /; push valid directory names, pop on . . , ignore . and empty segments. **Target:** Time $O(n)$, Space $O(n)$

10. Decode String

Recognize: nested repeated patterns like 3[a2[c]] **Hint:** Stack of (count, string-so-far); on], pop and repeat the substring, append to previous level. **Target:** Time $O(n)$, Space $O(n)$

11. Asteroid Collision

Recognize: collisions resolved by direction and size -> stack **Hint:** Stack holds surviving positive asteroids; a negative one pops smaller positives until it's absorbed or survives. **Target:** Time $O(n)$, Space $O(n)$

12. Basic Calculator II

Recognize: expression evaluation with operator precedence **Hint:** Stack of signed numbers; apply $*$ / $/$ immediately by popping and combining, defer $+$ / $-$ by pushing signed values. **Target:** Time $O(n)$, Space $O(n)$

13. Remove All Adjacent Duplicates in String

Recognize: cancel adjacent equal pairs **Hint:** Stack; pop if top equals current character, else push. **Target:** Time $O(n)$, Space $O(n)$

14. Online Stock Span

Recognize: streaming "span" of consecutive smaller-or-equal prices **Hint:** Monotonic decreasing stack of (price, span); pop and accumulate span while stack top's price $\leq$ current. **Target:** Time $O(1)$ amortized, Space $O(n)$

15. Implement Queue using Stacks

Recognize: FIFO built from LIFO structures **Hint:** Two stacks — push to `in`, and when `out` is empty, transfer everything from `in` to `out` for popping. **Target:** Time $O(1)$ amortized, Space $O(n)$

Self-check: Which problems used the stack for direct matching, and which used it as a "monotonic" structure? What made you choose increasing vs decreasing monotonic stacks?

Chapter 4: Linked List

Core pattern: pointer manipulation, dummy node, reversal, cycle detection

1. Reverse Linked List

Recognize: classic pointer reversal **Hint:** Iterative `prev/curr/next`; flip `curr.next` to `prev` each step. **Target:** Time $O(n)$, Space $O(1)$

2. Merge Two Sorted Lists

Recognize: merge step of merge sort, on lists **Hint:** Dummy node; compare heads, attach the smaller, advance that list's pointer. **Target:** Time $O(n+m)$, Space $O(1)$

3. Reorder List

Recognize: interleave first half with reversed second half **Hint:** Find middle (fast/slow), reverse the second half, merge the two halves alternately. **Target:** Time $O(n)$, Space $O(1)$

4. Remove Nth Node From End of List

Recognize: one-pass removal from the end **Hint:** Advance a fast pointer n steps first, then move both fast and slow together until fast hits the end. **Target:** Time $O(n)$, Space $O(1)$

5. Copy List with Random Pointer

Recognize: deep copy with an extra pointer **Hint:** HashMap of old->new node in one pass, then wire `next/random` in a second pass. **Target:** Time $O(n)$, Space $O(n)$

6. Add Two Numbers

Recognize: digit-by-digit addition with carry **Hint:** Dummy node; sum corresponding digits plus carry, create new node with `sum % 10`, carry `sum / 10`. **Target:** Time $O(\max(n,m))$, Space $O(\max(n,m))$

7. Linked List Cycle

Recognize: detect a cycle **Hint:** Floyd's fast/slow pointer — if they meet, a cycle exists. **Target:** Time $O(n)$, Space $O(1)$

8. Find the Duplicate Number

Recognize: array indices as an implicit linked list **Hint:** Treat `nums[i]` as a "next pointer"; apply Floyd's cycle detection to find the entry point (the duplicate). **Target:** Time $O(n)$, Space $O(1)$

9. LRU Cache

Recognize: $O(1)$ get/put with eviction order **Hint:** HashMap (key -> node) + doubly linked list; move accessed node to front, evict from the tail. **Target:** Time $O(1)$ per op, Space $O(\text{capacity})$

10. Merge K Sorted Lists

Recognize: merge many sorted lists efficiently **Hint:** Min-heap of current head nodes across lists, or divide-and-conquer pairwise merging. **Target:** Time $O(n \log k)$, Space $O(k)$

11. Reverse Nodes in k-Group

Recognize: chunked reversal **Hint:** Reverse each k -sized segment iteratively, and stitch the reversed segments together — leave a leftover segment ($< k$) unreversed. **Target:** Time $O(n)$, Space $O(1)$

12. Sort List

Recognize: sort a linked list in $O(n \log n)$ **Hint:** Merge sort — split with fast/slow into halves, recursively sort, merge two sorted lists. **Target:** Time $O(n \log n)$, Space $O(\log n)$ recursion

13. Palindrome Linked List

Recognize: check palindrome property with $O(1)$ space **Hint:** Find middle, reverse second half, compare the two halves node by node. **Target:** Time $O(n)$, Space $O(1)$

14. Intersection of Two Linked Lists

Recognize: find where two lists merge **Hint:** Two pointers switch to the other list's head after reaching their own end — they meet at the intersection (or both hit null together). **Target:** Time $O(n+m)$, Space $O(1)$

15. Flatten a Multilevel Doubly Linked List

Recognize: nested list flattening -> DFS-like traversal **Hint:** When a node has a child, recursively flatten the child list and splice it in between the node and its next. **Target:** Time $O(n)$, Space $O(\text{depth})$ recursion

16. Swap Nodes in Pairs

Recognize: swap adjacent nodes **Hint:** Dummy node; iteratively rewire pointers two nodes at a time. **Target:** Time $O(n)$, Space $O(1)$

17. Rotate List

Recognize: rotate list right by k **Hint:** Find length, connect tail to head to make it circular, break the new cycle at position $\text{length} - k \% \text{length}$. **Target:** Time $O(n)$, Space $O(1)$

18. Design Linked List

Recognize: implement full list API from scratch **Hint:** Maintain head (and optionally tail) pointers and a size counter; implement `get/addAtHead/addAtTail/addAtIndex/deleteAtIndex` directly on node pointers. **Target:** Time $O(n)$ worst case per op, Space $O(n)$

Self-check: Which problems needed the fast/slow pointer trick? Which needed a dummy node to simplify edge cases? Where did you need $O(1)$ space instead of a HashMap?

Chapter 5: Binary Search

Core pattern: $O(\log n)$ search over a sorted (or monotonic) space

1. Binary Search

Recognize: the base template **Hint:** $\text{mid} = (l+r)//2$; compare `nums[mid]` to target, shrink to the correct half. **Target:** Time $O(\log n)$, Space $O(1)$

2. Search a 2D Matrix

Recognize: row-and-column sorted matrix, treat as 1D **Hint:** Binary search over $0 \dots \text{rows} \cdot \text{cols} - 1$; convert `mid` to `row = mid/cols`, `col = mid%cols`. **Target:** Time $O(\log(m \cdot n))$, Space $O(1)$

3. Koko Eating Bananas

Recognize: “binary search on the answer” — minimize a feasible speed/capacity **Hint:** Binary search the eating speed; a feasibility check function tells you if that speed finishes in time. **Target:** Time $O(n \log m)$, Space $O(1)$

4. Find Minimum in Rotated Sorted Array

Recognize: rotated sorted array **Hint:** Compare `nums[mid]` to `nums[right]` to pick which half holds the minimum. **Target:** Time $O(\log n)$, Space $O(1)$

5. Search in Rotated Sorted Array

Recognize: target search in rotated sorted array **Hint:** Identify the sorted half at each step, then check if target falls in that half's range. **Target:** Time $O(\log n)$, Space $O(1)$

6. Search in Rotated Sorted Array II

Recognize: like above but with duplicates **Hint:** When `nums[l] == nums[mid] == nums[r]`, you can't tell which half is sorted — shrink both ends by one and continue. **Target:** Time $O(\log n)$ avg, $O(n)$ worst, Space $O(1)$

7. Time Based Key-Value Store

Recognize: query "largest timestamp $\leq$ given time" per key **Hint:** Store a list of (timestamp, value) per key; binary search that list for the right timestamp. **Target:** Time $O(\log n)$ per get, Space $O(n)$

8. Median of Two Sorted Arrays

Recognize: partition-based binary search across two arrays **Hint:** Binary search a partition point on the smaller array such that $\max(\text{left parts}) \leq \min(\text{right parts})$. **Target:** Time $O(\log(\min(n, m)))$, Space $O(1)$

9. Find First and Last Position of Element in Sorted Array

Recognize: boundary search **Hint:** Run binary search twice — once biased to find the leftmost occurrence, once for the rightmost. **Target:** Time $O(\log n)$, Space $O(1)$

10. Capacity To Ship Packages Within D Days

Recognize: binary search on the answer (capacity) **Hint:** Binary search over possible capacities; a greedy feasibility check counts days needed for a given capacity. **Target:** Time $O(n \log(\text{sum}))$, Space $O(1)$

11. Split Array Largest Sum

Recognize: binary search on the answer (max subarray sum) **Hint:** Same feasibility-check pattern as the shipping problem — binary search the max allowed subarray sum. **Target:** Time $O(n \log(\text{sum}))$, Space $O(1)$

12. Find Peak Element

Recognize: find a local maximum in $O(\log n)$ **Hint:** Compare `nums[mid]` to `nums[mid+1]`; move toward the side that's increasing (a peak is guaranteed there). **Target:** Time $O(\log n)$, Space $O(1)$

13. Search Insert Position

Recognize: binary search returning insertion index **Hint:** Standard binary search; when the loop ends, `left` is the correct insertion index. **Target:** Time $O(\log n)$, Space $O(1)$

14. Sqrt(x)

Recognize: binary search over the answer space **Hint:** Search $0 \dots x$ for the largest n where $n * n \leq x$. **Target:** Time $O(\log x)$, Space $O(1)$

15. Kth Smallest Element in a Sorted Matrix

Recognize: binary search over value range, not index range **Hint:** Binary search on the value; count how many matrix elements are $\leq$ mid using the matrix's sorted structure, compare count to k. **Target:** Time $O(n \log(\max - \min))$, Space $O(1)$

Self-check: Which problems searched over indices, and which searched over a value/answer space? What's the general shape of a "feasibility check" function?

Chapter 6: Sliding Window

Core pattern: contiguous subarray/substring optimization, fixed or variable window

1. Best Time to Buy and Sell Stock

Recognize: implicit window with a running minimum **Hint:** Track running min price; compare profit at each step. **Target:** Time $O(n)$, Space $O(1)$

2. Longest Substring Without Repeating Characters

Recognize: variable window, no repeats allowed **Hint:** HashSet/HashMap of last-seen index; shrink left pointer past the duplicate when one appears. **Target:** Time $O(n)$, Space $O(\min(n, \text{charset}))$

3. Longest Repeating Character Replacement

Recognize: variable window with a "budget" of allowed replacements **Hint:** Track count of the most frequent char in window; shrink when $\text{windowSize} - \text{maxFreq} > k$. **Target:** Time $O(n)$, Space $O(26)$

4. Permutation in String

Recognize: fixed window matching a character-frequency profile **Hint:** Window size = length of s1; compare 26-length count arrays as the window slides. **Target:** Time $O(n)$, Space $O(26)$

5. Minimum Window Substring

Recognize: variable window covering all target characters **Hint:** Expand right until all target chars are satisfied, then shrink left while still valid, tracking the minimum length window. **Target:** Time $O(n)$, Space $O(k)$

6. Sliding Window Maximum

Recognize: max in every fixed window, efficiently **Hint:** Monotonic decreasing deque of indices; pop from the back while smaller than the new element, pop from the front when out of window range. **Target:** Time $O(n)$, Space $O(k)$

7. Maximum Sum Subarray of Size K

Recognize: fixed window sum **Hint:** Compute the first window sum, then slide by subtracting the leaving element and adding the new one. **Target:** Time $O(n)$, Space $O(1)$

8. Fruit Into Baskets

Recognize: longest window with at most 2 distinct types **Hint:** Same as "at most K distinct" pattern with $K=2$; HashMap of counts, shrink when distinct count exceeds 2. **Target:** Time $O(n)$, Space $O(1)$

9. Find All Anagrams in a String

Recognize: fixed window matching a frequency profile, collect all matches **Hint:** Same as Permutation in String but record every valid start index instead of stopping at the first. **Target:** Time $O(n)$, Space $O(26)$

10. Longest Subarray of 1's After Deleting One Element

Recognize: window allowing at most one “bad” element (a zero) **Hint:** Track zero count in window; shrink when zero count exceeds 1. Answer is window length minus 1 (since one element must be deleted). **Target:** Time $O(n)$, Space $O(1)$

11. Minimum Size Subarray Sum

Recognize: shortest window with sum $\geq$ target **Hint:** Expand right accumulating sum; shrink left while sum $\geq$ target, tracking the minimum window length. **Target:** Time $O(n)$, Space $O(1)$

12. Subarrays with K Different Integers

Recognize: exact-K distinct count via “at most” trick **Hint:** Answer = $\text{atMost}(K) - \text{atMost}(K-1)$, where atMost counts subarrays with at most K distinct integers using a sliding window. **Target:** Time $O(n)$, Space $O(n)$

13. Max Consecutive Ones III

Recognize: window allowing at most k zeros **Hint:** Track zero count; shrink left while zero count $> k$, track the max window length. **Target:** Time $O(n)$, Space $O(1)$

Self-check: Which problems used a fixed-size window vs a variable-size one? What's the general shrink condition pattern you noticed across variable windows?

Chapter 7: Trees

Core pattern: DFS (preorder/inorder/postorder), BFS, recursion, tree construction

1. Invert Binary Tree

Recognize: mirror the tree **Hint:** Recursively swap left and right children at every node. **Target:** Time $O(n)$, Space $O(h)$

2. Maximum Depth of Binary Tree

Recognize: height computation **Hint:** $1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$, base case null $\rightarrow 0$. **Target:** Time $O(n)$, Space $O(h)$

3. Diameter of Binary Tree

Recognize: longest path between any two nodes **Hint:** Recursive height function that also updates a global max using $\text{leftHeight} + \text{rightHeight}$ at each node. **Target:** Time $O(n)$, Space $O(h)$

4. Balanced Binary Tree

Recognize: height-balance check **Hint:** Recursive height function returning -1 as a sentinel the moment imbalance is detected, to short-circuit. **Target:** Time $O(n)$, Space $O(h)$

5. Same Tree

Recognize: structural + value equality **Hint:** Recursively compare both trees' current nodes and both children pairs simultaneously. **Target:** Time $O(n)$, Space $O(h)$

6. Subtree of Another Tree

Recognize: does a smaller tree appear anywhere inside a bigger one **Hint:** For every node in the main tree, run a "same tree" check against the target subtree. **Target:** Time $O(n \cdot m)$, Space $O(h)$

7. Lowest Common Ancestor of a BST

Recognize: BST property lets you skip a full search **Hint:** Go left if both target values are less than the current node, right if both are greater; otherwise the current node is the LCA. **Target:** Time $O(h)$, Space $O(1)$

8. Binary Tree Level Order Traversal

Recognize: level-by-level output -> BFS **Hint:** Queue-based BFS; snapshot the queue's current size at the start of each level to know how many nodes belong to it. **Target:** Time $O(n)$, Space $O(n)$

9. Binary Tree Right Side View

Recognize: last node visible at each level **Hint:** BFS level order, keep the last node processed per level (or DFS right-first, recording the first node seen at each depth). **Target:** Time $O(n)$, Space $O(n)$

10. Count Good Nodes in Binary Tree

Recognize: track a running max along root-to-node paths **Hint:** DFS carrying the max value seen so far on the path; increment count when current node $\geq$ that max. **Target:** Time $O(n)$, Space $O(h)$

11. Validate Binary Search Tree

Recognize: BST property must hold globally, not just locally **Hint:** DFS passing down a valid (min, max) range for each node, tightening it as you recurse. **Target:** Time $O(n)$, Space $O(h)$

12. Kth Smallest Element in a BST

Recognize: inorder traversal gives sorted order in a BST **Hint:** Iterative inorder traversal with an explicit stack; stop and return once you've popped the kth node. **Target:** Time $O(h+k)$, Space $O(h)$

13. Construct Binary Tree from Preorder and Inorder Traversal

Recognize: rebuild a tree from two traversal orders **Hint:** `preorder[0]` is the root; find it in inorder to split left/right subtree ranges, recurse. Use a HashMap for $O(1)$ index lookup in inorder. **Target:** Time $O(n)$, Space $O(n)$

14. Binary Tree Maximum Path Sum

Recognize: path can start/end anywhere, not just root-to-leaf **Hint:** DFS returns the best single-direction path gain from each node (for the parent to use), but also updates a global max considering `left + right + node` (a "split" path). **Target:** Time $O(n)$, Space $O(h)$

15. Serialize and Deserialize Binary Tree

Recognize: convert tree $\leftrightarrow$ string, preserving structure **Hint:** Preorder DFS with explicit null markers (e.g. "#"); deserialize by consuming tokens in the same order via a queue/iterator. **Target:** Time $O(n)$, Space $O(n)$

16. Path Sum II

Recognize: find all root-to-leaf paths matching a sum **Hint:** DFS backtracking, maintaining the current path list and remaining sum; append a copy of the path when a leaf hits sum 0. **Target:** Time $O(n^2)$ worst (due to path copies), Space $O(h)$

17. Symmetric Tree

Recognize: mirror symmetry check **Hint:** Recursive helper comparing `left.left` vs `right.right`, and `left.right` vs `right.left`. **Target:** Time $O(n)$, Space $O(h)$

18. Flatten Binary Tree to Linked List

Recognize: turn the tree into a right-skewed “linked list” in preorder order **Hint:** Recursively flatten left and right subtrees first, then attach the flattened left subtree between the node and its right subtree. **Target:** Time $O(n)$, Space $O(h)$

19. Populating Next Right Pointers in Each Node

Recognize: connect same-level nodes horizontally **Hint:** BFS level order works generally; for $O(1)$ space (perfect binary tree), use the already-established next pointers of the previous level to traverse and link the current level. **Target:** Time $O(n)$, Space $O(1)$ (perfect tree version)

20. Vertical Order Traversal of a Binary Tree

Recognize: group nodes by column, respecting row and value order **Hint:** BFS/DFS tracking (`column`, `row`, `value`) for every node; group by column, then sort each group by row (and value for ties at the same position). **Target:** Time $O(n \log n)$, Space $O(n)$

Self-check: Which problems needed BFS instead of DFS, and why? Where did you pass state down the recursion (top-down) vs return state up (bottom-up)?

Chapter 8: Tries

Core pattern: prefix tree — string prefix matching, autocomplete-style problems

1. Implement Trie (Prefix Tree)

Recognize: the base data structure **Hint:** Each node holds a children map (or array[26]) and an `isEnd` flag; insert/search/startsWith all walk character by character. **Target:** Time $O(L)$ per op, Space $O(\text{total characters inserted})$

2. Design Add and Search Words Data Structure

Recognize: trie search with wildcard support **Hint:** Standard trie, but on a `.` wildcard character, branch into DFS across all children at that node. **Target:** Time $O(26^L)$ worst case (many wildcards), Space $O(\text{total characters})$

3. Word Search II

Recognize: find many words in a grid simultaneously $\rightarrow$ trie + DFS **Hint:** Build a trie of all target words, then DFS the grid guided by the trie — this prunes paths that can't lead to any word, and lets you find multiple words in one pass. **Target:** Time $O(m \cdot n \cdot 4^L)$, Space $O(\text{total characters in words})$

4. Replace Words

Recognize: replace each word with its shortest matching root **Hint:** Insert all roots into a trie; for each word, walk the trie until you hit an `isEnd` node — that's the shortest matching root. **Target:** Time $O(\text{total characters})$, Space $O(\text{total root characters})$

5. Maximum XOR of Two Numbers in an Array

Recognize: maximize XOR -> bit-level trie **Hint:** Build a binary trie of each number's bits (MSB to LSB); for each number, greedily walk the trie choosing the opposite bit at each level to maximize XOR. **Target:** Time $O(n \cdot 32)$, Space $O(n \cdot 32)$

6. Longest Word in Dictionary

Recognize: longest word buildable one character at a time from existing words **Hint:** Insert all words into a trie; DFS/BFS only through nodes where every prefix along the way is also a complete word (`isEnd` at every step). **Target:** Time $O(\text{total characters})$, Space $O(\text{total characters})$

7. Design Search Autocomplete System

Recognize: prefix-based suggestions ranked by frequency **Hint:** Trie storing full sentences with frequency counts at leaf/end nodes; at each keystroke, DFS from the current prefix node (or maintain a small heap) to get the top suggestions. **Target:** Time $O(\text{prefix length} + \text{matches})$, Space $O(\text{total sentence characters})$

8. Palindrome Pairs

Recognize: pairs of words that concatenate into a palindrome **Hint:** Build a trie of reversed words storing indices; for each word, check palindrome splits against the trie (a HashMap-based approach is often simpler to implement). **Target:** Time $O(n \cdot L^2)$, Space $O(n \cdot L)$

9. Stream of Characters

Recognize: streaming suffix matching against a dictionary **Hint:** Build a trie of reversed dictionary words; maintain a buffer of the stream so far, and check the trie backwards from the most recent character each time. **Target:** Time $O(L)$ per query, Space $O(\text{total characters in dictionary})$

Self-check: Where did a trie beat a plain HashMap, and why? Which problems combined a trie with DFS/backtracking on top?

Chapter 9: Heap / Priority Queue

Core pattern: min-heap/max-heap for top-k, scheduling, running median

1. Kth Largest Element in a Stream

Recognize: maintain the kth largest as elements keep arriving **Hint:** Min-heap of size k; push new value, pop if size exceeds k — the heap's top is the answer. **Target:** Time $O(\log k)$ per insert, Space $O(k)$

2. Last Stone Weight

Recognize: repeatedly combine the two largest values **Hint:** Max-heap (or min-heap of negated values); pop two largest, push back their difference. **Target:** Time $O(n \log n)$, Space $O(n)$

3. K Closest Points to Origin

Recognize: top-k by distance **Hint:** Max-heap of size k by distance (evict the farthest when size exceeds k), or Quickselect for $O(n)$ average. **Target:** Time $O(n \log k)$, Space $O(k)$

4. Kth Largest Element in an Array

Recognize: classic top-k selection **Hint:** Min-heap of size k, or Quickselect (partition-based) for $O(n)$ average time. **Target:** Time $O(n \log k)$ heap / $O(n)$ avg Quickselect, Space $O(k)$

5. Task Scheduler

Recognize: schedule with a cooldown constraint -> greedy + max-heap **Hint:** Max-heap by remaining frequency; greedily fill each cycle with the most frequent available tasks, or use the closed-form formula based on max frequency and cooldown. **Target:** Time $O(n \log 26)$, Space $O(26)$

6. Design Twitter

Recognize: merge k sorted (by time) feeds **Hint:** HashMap of user -> tweet list (with timestamps) and user -> followers; use a max-heap to merge the most recent tweets across followed users. **Target:** Time $O(k \log(\text{followers}))$ per feed fetch, Space $O(n)$

7. Find Median from Data Stream

Recognize: maintain a running median efficiently **Hint:** Two heaps — max-heap for the lower half, min-heap for the upper half; keep their sizes balanced (differ by at most 1). **Target:** Time $O(\log n)$ per insert, Space $O(n)$

8. Merge K Sorted Lists

Recognize: merge many sorted sequences **Hint:** Min-heap of the current head node from each list; pop the smallest, push its successor. **Target:** Time $O(n \log k)$, Space $O(k)$

9. Top K Frequent Elements

Recognize: top-k by frequency **Hint:** Min-heap of size k by frequency (evict smallest when size exceeds k), or bucket sort for $O(n)$. **Target:** Time $O(n \log k)$ heap, Space $O(n)$

10. Reorganize String

Recognize: rearrange so no two adjacent characters are equal -> greedy + max-heap **Hint:** Max-heap by frequency; always place the currently most frequent character, holding back the previously placed one for one step to avoid adjacency. **Target:** Time $O(n \log 26)$, Space $O(26)$

11. Meeting Rooms II

Recognize: track concurrently active intervals **Hint:** Min-heap of end times; for each meeting, if the earliest-ending room is free (its end $\leq$ current start), reuse it, else add a new room. **Target:** Time $O(n \log n)$, Space $O(n)$

12. Minimum Cost to Connect Sticks

Recognize: repeatedly combine the two cheapest items **Hint:** Min-heap; repeatedly pop the two smallest sticks, combine (sum as cost), push the sum back. **Target:** Time $O(n \log n)$, Space $O(n)$

13. Single-Threaded CPU

Recognize: process available tasks by shortest processing time first **Hint:** Sort tasks by enqueue time; maintain a min-heap of currently available tasks ordered by (processing time, original index). **Target:** Time $O(n \log n)$, Space $O(n)$

14. IPO

Recognize: greedily pick the most profitable affordable project, repeatedly **Hint:** Sort projects by capital requirement; use a pointer to add all currently affordable projects into a max-heap by profit, then repeatedly take the top. **Target:** Time $O(n \log n)$, Space $O(n)$

15. Process Tasks Using Servers

Recognize: simulate task assignment across free/busy servers over time **Hint:** Two min-heaps — free servers ordered by (weight, index), busy servers ordered by (finish time, index); advance time by popping busy servers that finish before the next task arrives. **Target:** Time $O(n \log n)$, Space $O(n)$

Self-check: Which problems needed two heaps working together? Where did a heap beat sorting the whole array upfront, and why?

Chapter 10: Backtracking

Core pattern: explore a decision tree, prune invalid branches — combination/permutation/constraint problems

1. Subsets

Recognize: all possible subsets **Hint:** DFS with an include/exclude decision at each index, or build iteratively by doubling the result list. **Target:** Time $O(n \cdot 2^n)$, Space $O(n)$

2. Combination Sum

Recognize: combinations summing to target, elements reusable **Hint:** DFS trying each candidate from the current index onward; don't advance the index when reusing an element, backtrack when the running sum exceeds target. **Target:** Time $O(2^{\text{target}})$ worst, Space $O(\text{target})$

3. Combination Sum II

Recognize: like above but no reuse, and input has duplicates **Hint:** Sort first; always advance the index (no reuse), and skip duplicate values at the same recursion depth. **Target:** Time $O(2^n)$, Space $O(n)$

4. Permutations

Recognize: all orderings of a set **Hint:** DFS swapping elements into place at each recursion level, or maintain a `used[]` boolean array while building the permutation. **Target:** Time $O(n \cdot n!)$, Space $O(n)$

5. Permutations II

Recognize: permutations with duplicate elements **Hint:** Sort first; skip a duplicate value at the same recursion depth unless its earlier duplicate has already been used. **Target:** Time $O(n \cdot n!)$, Space $O(n)$

6. Subsets II

Recognize: subsets with duplicate elements **Hint:** Sort first; when doing include/exclude DFS, skip a duplicate value if the previous identical value wasn't included at this depth. **Target:** Time $O(n \cdot 2^n)$, Space $O(n)$

7. Word Search

Recognize: path matching in a grid -> DFS + backtracking **Hint:** DFS from each starting cell, mark visited cells temporarily, restore on backtrack. **Target:** Time $O(m \cdot n \cdot 4^L)$, Space $O(L)$

8. Palindrome Partitioning

Recognize: all ways to split a string into palindromic substrings **Hint:** DFS trying every possible prefix at each position; only recurse into the remainder if the prefix is itself a palindrome. **Target:** Time $O(n \cdot 2^n)$, Space $O(n)$

9. Letter Combinations of a Phone Number

Recognize: build strings letter by letter from digit choices **Hint:** DFS through each digit's mapped letters, appending one character per recursion level. **Target:** Time $O(4^n)$, Space $O(n)$

10. N-Queens

Recognize: place N queens with no attacking pairs **Hint:** DFS row by row, choosing a column per row; track occupied columns and both diagonals using sets for $O(1)$ conflict checks. **Target:** Time $O(n!)$, Space $O(n)$

11. N-Queens II

Recognize: same as N-Queens, but count solutions only **Hint:** Identical DFS to N-Queens, but increment a counter at a complete placement instead of storing the board. **Target:** Time $O(n!)$, Space $O(n)$

12. Sudoku Solver

Recognize: constraint satisfaction over a 9x9 grid **Hint:** DFS trying digits 1–9 for each empty cell in order, validating row/column/box constraints before placing, backtrack on dead ends. **Target:** Time $O(9^{(\text{empty cells})})$ worst case, Space $O(1)$ extra

13. Combination Sum III

Recognize: choose k numbers from 1–9 summing to target **Hint:** DFS similar to Combination Sum, but always advancing the index (no reuse) and pruning when the remaining count of numbers can't possibly reach the target. **Target:** Time $O(C(9, k))$, Space $O(k)$

14. Generate Parentheses

Recognize: valid parenthesis combinations **Hint:** DFS tracking open/close counts; add (if open < n, add) if close < open. **Target:** Time $O(4^n / \text{sqrt}(n))$, Space $O(n)$

15. Restore IP Addresses

Recognize: split a string into 4 valid IP segments **Hint:** DFS trying segment lengths 1–3 at each position; validate each segment is 0–255 with no invalid leading zero, backtrack if not. **Target:** Time $O(3^4)$, Space $O(1)$ extra

16. Word Break II

Recognize: produce every valid sentence decomposition **Hint:** DFS/backtracking trying every dictionary word as a prefix at each position, with memoization on the remaining substring to avoid recomputation. **Target:** Time $O(2^n)$ worst, Space $O(2^n)$ for memo

17. Matchsticks to Square

Recognize: partition into 4 equal-sum groups **Hint:** Sort sticks descending (fail fast); DFS distributing sticks into 4 buckets, pruning any bucket that exceeds the target side length. **Target:** Time $O(4^n)$, Space $O(n)$

Self-check: Which problems needed sorting first to enable duplicate-skipping? What's the general shape of a backtracking function (choose -> recurse -> un-choose)?

Overall Timeline Reminder

Week	Topic
1	Arrays + Two Pointers
2	Stack + Linked List
3	Binary Search + Sliding Window
4	Trees
5	Tries + Heap
6	Backtracking + Full Review

Solve each problem yourself for 25–30 minutes before checking a hint. Write a one-line summary of your approach after each solve — that's what builds real pattern recognition, not memorization.